

Special Participation E 1

Edward Zhang

December 2025

2. Factorizing a matrix whose singular values are 1

Write SVDs

» Ask ChatGPT $W_1 = U_1 \Sigma_1 V_1^\top, \quad W_2 = U_2 \Sigma_2 V_2^\top.$

If we align the bases appropriately (so that $V_1 = U_2$ on the relevant subspace), then on that k -dimensional subspace

$$W_2 W_1 \approx U_2 \Sigma_2 V_2^\top U_1 \Sigma_1 V_1^\top \sim \Sigma_2 \Sigma_1.$$

Figure 1: Caption

1 Asking Technique

If there are anything confused in ChatGPT's answer, I would highlighted it, and click "Ask ChatGPT." Then, ChatGPT would start explaining what I have confused about, which is convenient as shown in Figure 1.

2

There are still some parts that ChatGPT's explaining are confusing. For example, It says:

$$U_2 \Sigma_2 V_2^T U_1 \Sigma_1 V_1^T \sim \Sigma_2 \Sigma_1$$

The symbol $\sim$ is confusing since it is not defined. It seems that the explanation is not mathematical rigorous but leaning on the intuition.

3 Conclusion

As I worked through this problem, I clarified several concepts: I now understand teacher forcing as feeding the ground-truth previous token into the decoder, and why the per-step loss is written as $-\log p_\theta(y_t \mid h_t)$ even though the gradient still flows back through $h_{t-1}, y_{t-1}, \dots$ via BPTT. I also connected weight decay to adding an ℓ_2 penalty and saw how, in this low-rank factorization setting, regularizing $\|W_1\|_F^2 + \|W_2\|_F^2$ induces a preference for factorizations where W_2W_1 is an identity on the top- k subspace and the singular values of W_1 and W_2 pair as σ_i and $1/\sigma_i$, leading to approximately orthonormal columns. My questioning style was very step-by-step and “why-focused”: I kept asking for unpacking of specific sentences, challenging hidden assumptions (like aligning V_1 and U_2), and requesting intuitive explanations of each algebraic step rather than just accepting the final formula.

Training time explanation

User

[File]: [file_00000000427071fd95d8228f935b1173-sanitized.png](#)

[File]: [file_00000000bd6471fd85520e13fc49d0ce-sanitized.png](#)

Why the solution to (b) is <sos>, I, see, a, dog?

ChatGPT

```
from PIL import Image, ImageOps, ImageFilter, ImageEnhance, ImageStat, ImageChops,
ImageFilter
img1 = Image.open('/mnt/data/bd05d18e-412a-44ed-ae49-a6859812ced5.png')
display(img1.resize((900,600)))
```

ChatGPT

```
img2 = Image.open('/mnt/data/dcc3d780-b904-4562-8b62-c877aec2d35b.png')
display(img2.resize((900,200)))
```

ChatGPT

Because the question is about **training time**, not test time.

- The correct English translation of the Spanish sentence is **"I see a dog"**.
- During training we use **teacher forcing**: at each decoder step we feed in the **ground-truth previous token**, not whatever the model happened to predict.
- We also prepend the special start-of-sequence token `<sos>`.

So the inputs to the decoder RNN at each time step are:

1. $w_1 = \text{<sos>}$
2. $w_2 = \text{"I"}$ (ground-truth first word)
3. $w_3 = \text{"see"}$
4. $w_4 = \text{"a"}$
5. $w_5 = \text{"dog"}$

The fact that the model **sampled** "I **saw** a dog" in the example is only describing what happens at test time; during training we still feed it the correct sequence "I see a dog", hence the solution `<sos>, I, see, a, dog`.

User

what is teacher forcing?

ChatGPT

Teacher forcing is a training trick for sequence models (RNNs, seq2seq, etc.).

Idea:

At decoding time step t , instead of feeding the model its **own** previous prediction $\hat{y}_{t-1}$, we feed it the **ground-truth** previous token y_{t-1} .

So for a target sequence $(y_1, y_2, \dots, y_T)$:

- Hidden update: $h_t = f(h_{t-1}, \text{text_embed}(y_{t-1}))$
- Prediction: $\hat{y}_t = g(h_t)$
- Loss at step t : $\mathcal{L}_t = -\log p_{\theta}(y_t \mid h_t)$

In your MT example:

- Step 1 input: `<sos>`
- Step 2 input: **true** token "I" (not whatever the model predicted after `<sos>`)
- Step 3 input: "see"

- Step 4 input: "a"
- Step 5 input: "dog"

****Why use it?****

- Makes training easier and faster to converge (the model always conditions on correct history).

****Downside (exposure bias):****

- At test time you **don't** have the ground truth, so you feed back its own predictions. The model has never been trained on its own mistakes, so errors can compound.

People sometimes mitigate this with scheduled sampling, professor forcing, etc., but plain teacher forcing is exactly "feed the true previous token during training."

User

Why is the loss $-\log p_{\theta}(y_t | h_t)$? Why does the loss only depend on the y_t and h_t instead of including h_{t-1} and y_{t-1} ? This would only train the function g without training the function f , right?

ChatGPT

Good questions – this is exactly where the notation hides what's really going on.

1. Why is the loss $-\log p_{\theta}(y_t | h_t)$?

At each time step t , the model outputs a ****distribution over the vocabulary****:

$$p_{\theta}(y | h_t) = \text{softmax}(W h_t + b)_y$$

We observe a ****single correct token**** y_t from the dataset. Training by maximum likelihood means: ***make the probability of the correct token as large as possible***.

So for step t , the negative log-likelihood loss is

$$\mathcal{L}_t(\theta) = -\log p_{\theta}(y_t | h_t)$$

This is just the usual cross-entropy between a one-hot target (at y_t) and the predicted distribution.

2. Why doesn't the loss explicitly include h_{t-1} and y_{t-1} ?

Short answer: ****they're inside h_t ****

The RNN recurrence is something like

$$h_t = f_{\theta}(h_{t-1}, \text{embed}(y_{t-1}), \text{context})$$

So h_t is ***computed from*** h_{t-1} , y_{t-1} , etc. Writing $\mathcal{L}_t = -\log p_{\theta}(y_t | h_t)$ is shorthand; more explicitly you could write

$$\mathcal{L}_t(\theta)$$

```
= -\log p_{\theta}(\mathbf{y}_t \mid \mathbf{h}_t, \mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \dots)
= -\log p_{\theta}(\mathbf{y}_t \mid \mathbf{h}_t, \mathbf{f}_{\theta}(\mathbf{h}_{t-1}, \mathbf{y}_{t-1}))
\]
```

So the loss **does** depend on $(\mathbf{h}_{t-1})$, $(\mathbf{y}_{t-1})$, $(\mathbf{h}_{t-2}, \mathbf{y}_{t-2}, \dots)$ – just indirectly through the hidden state.

3. Doesn't this only train (g) and not (f) ?

No – both get trained because of **backpropagation through time (BPTT)**.

Think of the model as two parts:

- (f_{θ}) : the recurrence that updates $(\mathbf{h}_t)$ from $(\mathbf{h}_{t-1})$ and the input token
- (g_{θ}) : the readout that maps $(\mathbf{h}_t)$ to a distribution over tokens

The loss depends on (θ) in **two** ways:

1. Directly through (g_{θ}) (the output layer),
2. Indirectly through $(\mathbf{h}_t)$, which is produced by repeatedly applying (f_{θ}) .

When you take the gradient:

```
\[
\frac{\partial \mathcal{L}_t}{\partial \theta}
= \underbrace{\frac{\partial \mathcal{L}_t}{\partial g_{\theta}}}_{\text{updates } g}
+ \underbrace{\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \theta}}_{\text{updates } f \text{ via BPTT}}
\]
```

And $(\frac{\partial \mathbf{h}_t}{\partial \theta})$ itself expands through the recurrence:

```
\[
\frac{\partial \mathbf{h}_t}{\partial \theta}
= \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \theta} + \frac{\partial \mathbf{h}_t}{\partial \theta} \Big|_{\text{local}}
\]
```

so gradients flow **back in time** through $(\mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \dots)$. That's exactly what "backpropagation through time" does.

So:

- The term $(-\log p_{\theta}(\mathbf{y}_t \mid \mathbf{h}_t))$ is written only with $(\mathbf{y}_t)$ and $(\mathbf{h}_t)$ for notational simplicity.
- But because $(\mathbf{h}_t)$ is a **function** of all past tokens and states, the loss at time (t) sends gradient signals through the whole chain, training **both** the transition function (f) and the readout (g) .

If you like, you can mentally read the loss as:

```
\[
\mathcal{L}_t(\theta) = -\log p_{\theta}(\mathbf{y}_t \mid \mathbf{y}_{<t}, \mathbf{x})
\]
```

and teacher forcing is what lets us compute that by feeding the true $(\mathbf{y}_{<t})$ into the recurrence to produce $(\mathbf{h}_t)$.

User

What is weight decay?

ChatGPT

Weight decay is a regularization trick that gently pushes network weights toward zero while training, to reduce overfitting.

As a loss term (L2 regularization)

You change your loss from just

$$\sum \text{loss}(\text{data}; \theta)$$

to

$$\sum \text{loss}(\text{data}; \theta) + \frac{\lambda}{2} \|\theta\|_2^2,$$

where θ are the weights and $\lambda > 0$ controls how strong the penalty is. The model is now trying to fit the data **and** keep the weights small.

Why it's called "decay"

For plain SGD, the gradient of the regularizer is $\lambda \theta$, so the update becomes

$$\theta_{t+1} = \theta_t - \eta \left(\nabla_{\theta} \text{loss}(\text{data}; \theta_t) + \lambda \theta_t \right) \\ = (1 - \eta \lambda) \theta_t - \eta \nabla_{\theta} \text{loss}(\text{data}; \theta_t).$$

That factor $(1 - \eta \lambda)$ multiplies the weights every step, so they **decay** toward zero over time, unless the data gradient pushes them away.

Intuition

- Encourages **smaller weights**, which usually means smoother, less extreme functions.
- Helps **prevent overfitting**, especially when the model is large.
- In optimizers like AdamW, "weight decay" is often implemented as this multiplicative shrink step, decoupled from the gradient, but the idea is the same: nudge parameters toward zero every step.

User

[File]: [file_00000000c4071fda0a8c57d169d40a5-sanitized.png](#)

[File]: [file_00000000c49071fda89662038bd1a234-sanitized.png](#)

I didn't understand this question (b), what does it mean to have an inductive bias towards finding a W_2 matrix with approximately orthonormal columns?

ChatGPT

Roughly, the question is asking:

> When we add that small weight-decay term $\lambda (\|W_1\|_F^2 + \|W_2\|_F^2)$, does the objective **prefer** solutions where the columns of W_2 look like an orthonormal basis?

Let me unpack the terms.

1. "Orthonormal columns" of W_2

Suppose $W_2 \in \mathbb{R}^{d \times k}$.

Its columns $\{w_1, \dots, w_k \in \mathbb{R}^d\}$ are **orthonormal** if

$$W_2^T W_2 = I_k,$$

i.e.

- each column has unit length: $\|w_i\|_2 = 1$,
- different columns are orthogonal: $w_i^T w_j = 0$ for $(i \neq j)$.

So “approximately orthonormal columns” means $W_2^T W_2 \approx I_k$.

2. What is “inductive bias” here?

Inductive bias = a **preference among many good solutions** that comes from the model/regularizer/optimizer, even though we never hard-coded that preference as a constraint.

In this problem:

- Without regularization ($\lambda = 0$), there can be **many** pairs (W_1, W_2) that give (almost) the same reconstruction error $\|X - W_2 W_1^T X\|_F^2$.
- When we add weight decay, we’re not just minimizing reconstruction error; we’re minimizing

$$\begin{aligned} \mathcal{L}(\lambda) &= \frac{1}{2} \|X - W_2 W_1^T X\|_F^2 \\ &+ \lambda \|W_1\|_F^2 \\ &+ \lambda \|W_2\|_F^2. \end{aligned}$$

Among all pairs (W_1, W_2) that reconstruct X similarly well, the objective now **prefers those with smaller Frobenius norms**.

So the question is:

> When you consider that extra “small-norm” preference, does it tend to pick out solutions where the columns of W_2 are (approximately) orthonormal?

In other words, does the combination of reconstruction + weight decay **implicitly** encourage W_2 to look like a matrix of (almost) orthonormal basis vectors, even though we never wrote the constraint $W_2^T W_2 = I$ anywhere?

That’s exactly what “an inductive bias towards finding a W_2 matrix with approximately orthonormal columns” means.

User

[File]: [file_00000000771471fdabf84fd4cfb966ff-sanitized.png](#)
what does the highlighted sentence mean?

ChatGPT

The sentence is unpacking what it means for $W_2 W_1^T$ to be **the projection onto the top- k singular subspace of X** .

> “ $W_2 W_1^T$... has to be the identity when restricted to the subspace spanned by the k largest singular vectors of X .”

Break it down:

- Let U_k be the matrix whose columns are the top k left singular vectors of X . The subspace they span is

$$S = \text{span}\{u_1, \dots, u_k\}.$$

- The best rank- k approximation of X is obtained by **projecting** onto this subspace. The projection matrix is $P = U_k U_k^{\text{top}}$.
- A projection matrix P has the property:
 - For any vector v **inside** the subspace S , $Pv = v$.
So, on that subspace, P does nothing—it's the **identity map** there.
 - For vectors orthogonal to S , $Pv = 0$.

So the highlighted sentence is saying:

> The product $W_2 W_1$ that minimizes the reconstruction loss must behave like this projector P . In particular, for any vector v that lies in the span of the top k singular vectors of X , we must have

$$W_2 W_1 v = v.$$

> That is, **if you only look at how $W_2 W_1$ acts on that k -dimensional subspace, it's exactly the identity matrix.**

“Restricted to the subspace” just means: ignore what $W_2 W_1$ does on other directions; on that special k -dimensional space, it acts like the identity.

User

[File]: [file_000000003c8471fdac5145fb1da76a30-sanitized.png](#)

Why W_1 has singular vector σ and W_2 has singular vector $1/\sigma$?

ChatGPT

Nice catch—this is about **singular values** (not vectors), and it comes from the fact that W_1 and W_2 are acting like pseudo-inverses of each other on the top- k subspace.

1. What property do we want?

From the solution text:

- To minimize the reconstruction term $\|X - W_2 W_1 X\|_F^2$, we want $W_2 W_1$ to behave like the **projection onto the top- k singular subspace of X** .
- That projector P has SVD

$$P = U_k I_k U_k^{\text{top}},$$
 so all its nonzero singular values are **1**.

Thus, on that k -dimensional subspace, we want

$$W_2 W_1 \approx I_k$$

so the **singular values of $W_2 W_1$ on that subspace are 1**.

2. Factorizing a matrix whose singular values are 1

Write SVDs

$$U$$

$$\begin{aligned} W_1 &= U_1 \Sigma_1 V_1^{\top}, \\ W_2 &= U_2 \Sigma_2 V_2^{\top}. \end{aligned}$$

If we align the bases appropriately (so that $(V_1 = U_2)$ on the relevant subspace), then on that (k) -dimensional subspace

$$W_2 W_1^{\top} \approx U_2 \Sigma_2 V_2^{\top} U_1^{\top} \Sigma_1 V_1^{\top} \approx \Sigma_2 \Sigma_1.$$

To make the product have singular values 1, we need

$$\Sigma_2 \Sigma_1 = I_k.$$

If we denote the diagonal entries (singular values) of (Σ_1) by (σ_i) , and those of (Σ_2) by $(\tilde{\sigma}_i)$, then this condition says

$$\sigma_i \tilde{\sigma}_i = 1 \quad \Rightarrow \quad \tilde{\sigma}_i = \frac{1}{\sigma_i}.$$

That's exactly where

- singular values of $(W_1) = (\sigma_i)$,
- singular values of $(W_2) = (1/\sigma_i)$

comes from.

3. Pseudoinverse viewpoint (same thing)

If (W_2) is (approximately) the **pseudoinverse** of (W_1) on that subspace, then:

- If $(W_1 = U \Sigma V^{\top})$, its pseudoinverse is $(W_1^+ = V \Sigma^{-1} U^{\top})$.
- The pseudoinverse has singular values $(1/\sigma_i)$.

So saying " (W_1) has singular values (σ_i) and (W_2) has singular values $(1/\sigma_i)$ " is the same as saying " (W_2) is (approximately) the pseudoinverse of (W_1) on that subspace," which ensures $(W_2 W_1^{\top} \approx I)$ there.

User

How do we align V_1 and U_2 ? How could you ensure that V_1 and U_2 are on the same basis since the SVD is unique for each W_1 and W_2 ?

ChatGPT

Great question—this is exactly the subtlety the solution is sweeping under the rug.

1. We are *choosing* (W_1, W_2) , we're not given them

In this problem (W_1) and (W_2) are **parameters we are optimizing**, not fixed matrices whose SVDs are predetermined. Among all factorizations that minimize the main loss, we're allowed to pick a particularly nice one to reason about.

So when I said "align the bases so that $(V_1 = U_2)$ ", I wasn't claiming some magic property of the SVD of *arbitrary* matrices; I was saying:

> Among all optimal (W_1, W_2) , we can w.l.o.g. consider a pair whose SVDs are chosen so their middle orthonormal factors match.

2. SVD is *not* completely unique

Even for a *fixed* matrix, the SVD has freedom:

- You can flip signs of corresponding columns in U and V .
- If some singular values are equal (e.g. all ones), you can rotate the corresponding singular vectors by *any* orthogonal matrix Q inside that subspace:

$$U \Sigma V^{\top} = (UQ) \Sigma (VQ)^{\top}$$

whenever Q commutes with Σ on that block. In particular, if the singular values on a k -dimensional block are all 1 (like for a projector / identity on that subspace), you have full rotational freedom in that block.

In our setting, we want $(W_2 W_1^{\top})$ to act like the identity on the top- k subspace, so all its nonzero singular values there are 1. That gives us exactly this rotational freedom in that k -dimensional subspace.

3. How to “align” V_1 and U_2

Think of the factorization on the relevant k -dimensional subspace as

$$\begin{aligned} W_1 &= U_1 \Sigma_1 R^{\top}, \\ W_2 &= R \Sigma_2 V_2^{\top}, \end{aligned}$$

where R is some $k \times k$ orthogonal matrix that we are free to choose (this is the same “rotation in the singular subspace” freedom).

Then the product is

$$W_2 W_1^{\top} = R \Sigma_2 V_2^{\top} U_1 \Sigma_1 R^{\top}.$$

By folding bases appropriately (and since we want the product to look like a projector with singular values 1 on that subspace), we can choose this common orthogonal matrix R so that the *right* singular vectors of W_1 and the *left* singular vectors of W_2 coincide:

$$V_1 \equiv R \equiv U_2.$$

So “aligning V_1 and U_2 ” just means:

> Use the SVD freedoms and our freedom to choose (W_1, W_2) to pick a representation where the intermediate orthonormal basis is shared.

Once we pick such a pair, the singular values condition $\Sigma_2 \Sigma_1 = I$ on that subspace forces the singular values of W_1 to be σ_i and those of W_2 to be $1/\sigma_i$, and then the Frobenius regularizer becomes

$$\begin{aligned} & \|W_1\|_F^2 + \|W_2\|_F^2 \\ &= \sum_{i=1}^k \left(\sigma_i^2 + \frac{1}{\sigma_i^2} \right). \end{aligned}$$

So: we’re not *proving* that arbitrary SVDs satisfy $V_1 = U_2$; we’re exploiting the non-uniqueness and optimization freedom to pick a convenient optimal factorization where they *do* match.